

Constrained Device Performance Benchmarking with the Implementation of PostQuantum Cryptography

Authors:

- Gregory Fitzgibbon
- Carlo Ottaviani

Motivation

Why Post Quantum Cryptography?

Current public-key cryptography relies on:

- RSA
- ECDSA
- Diffie-Hellman

Future quantum computers running Shor's algorithm may break these systems.

NIST launched the PQC standardization process to identify quantum-resistant alternatives.

Problem Statement

Most PQC benchmarking focuses on powerful computers.

However:

- IoT devices are resource-constrained
- Limited CPU
- Limited RAM
- Limited storage
- Limited power

→ Can modern PQC algorithms run efficiently on constrained devices?



Research Questions

RQ1

How do selected PQAs perform on constrained IoT devices compared to high-performance platforms?

RQ2

How do the obtained PQA results compare with findings from previous studies?

RQ3

How do PQA-based KEMs and signatures perform when integrated with TLS on constrained devices?

RQ4

What are the key advantages and limitations of PQAs on constrained devices?

Algorithms Evaluated

KEM Algorithms:

Family	Algorithm
Lattice	Kyber768
Code-Based	HQC-192
Code-Based	BIKE-L3
Code-Based	Classic McEliece-460896
Code-Based	Classic McEliece-460896f (SSF)

Signature Algorithms:

Family	Algorithm
Lattice	Dilithium3
Lattice	Falcon-1024
Hash-based	SPHINCS+-SHA256-192f-simple
Hash-based	SPHINCS+-SHA256-192s-simple

Experimental Setup

CONSTRAINED DEVICES

Raspberry Pi 4 (1 & 2):

- 1.5 GHz ARM Cortex-A72
- Quad Core
- 4 GB RAM
- Debian Bullseye 64-bit
- 64 GB microSD

Connected through:

- Netgear GS308 switch

CONTROL DEVICES

Desktop:

- Ryzen 9 5950X
- 16 cores / 32 threads
- 4.9 GHz
- 64 GB DDR4 RAM
- 1 TB NVMe SSD

Laptop:

- Intel i7-9750H
- 6 cores / 12 threads
- Up to 2.6 GHz
- 1 TB SSD

METHODOLOGY

NIST Security Level 3

Equivalent to 128-bit quantum security.

Metrics Measured

KEM: KeyGen, Encap, Decap

Signature: Keypair, Sign, Verify

Framework

Open Quantum Safe (liboqs).

SIKE Exclusion

Removed from liboqs after classical cryptanalytic attack compromised the algorithm.

KEM Benchmark Results: Desktop PC

Desktop PC (Ryzen 9 5950X)

ALGORITHM	KEYGEN	DECAPSULATION	ENCAPSULATION
Kyber768	41.97 µs (0.04197 ms)	57.23 µs (0.05723 ms)	49.23 µs (0.04923 ms)
HQC-192	3,753.14 µs (3.75314 ms)	11,350.02 µs (11.35002 ms)	7,518.21 µs (7.51821 ms)
Classic-McEliece-460896	83,262.43 µs (83.26243 ms)	55.32 µs (0.05532 ms)	22.93 µs (0.02293 ms)
Classic-McEliece-460896f	80.90 µs (0.08090 ms)	80,559.11 µs (80.55911 ms)	28,679.05 µs (28.67905 ms)
BIKE-L3	567.34 µs (0.56734 ms)	1,617.50 µs (1.61750 ms)	74.18 µs (0.07418 ms)

- Kyber768 was the fastest KEM on the PC, completing all operations in under 0.06 ms.
- Classic McEliece showed the highest latency, with key generation reaching 83.26 ms and decapsulation up to 80.56 ms.

KEM Benchmark Results: Laptop

Laptop (Intel i7-9750H)

ALGORITHM	KEYGEN	ENCAPSULATION	DECAPSULATION
Kyber768	152.74 μ s (0.15274 ms)	195.48 μ s (0.19548 ms)	234.65 μ s (0.23465 ms)
HQC-192	7,305.35 μ s (7.30535 ms)	15,238.48 μ s (15.23848 ms)	21,871.35 μ s (21.87135 ms)
Classic-McEliece-460896	222,979.55 μ s (222.97955 ms)	65.99 μ s (0.06599 ms)	133.45 μ s (0.13345 ms)
Classic-McEliece-460896f	216.23 μ s (0.21623 ms)	86,086.53 μ s (86.08653 ms)	218,209.23 μ s (218.20923 ms)
BIKE-L3	1,362.92 μ s (1.36292 ms)	170.39 μ s (0.17039 ms)	4,045.56 μ s (4.04556 ms)

- Kyber768 remained the fastest KEM on the laptop, with all operations completing in under 0.24 ms.
- Classic McEliece showed the highest latency, with key generation reaching 222.98 ms and decapsulation up to 218.21 ms.

KEM Benchmark Results: RPI 4 (I)

Raspberry PI 4 - I

ALGORITHM	KEYGEN	ENCAPSULATION	DECAPSULATION
Kyber768	114.74 μ s (0.11474 ms)	115.11 μ s (0.11511 ms)	86.48 μ s (0.08648 ms)
HQC-192	24,276.18 μ s (24.27618 ms)	48,478.83 μ s (48.47883 ms)	72,937.29 μ s (72.93729 ms)
Classic-McEliece-460896	1,907,859.17 μ s (1907.85917 ms)	472.19 μ s (0.47219 ms)	106,634.09 μ s (106.63409 ms)
Classic-McEliece-460896f	470.42 μ s (0.47042 ms)	106,598.62 μ s (106.59862 ms)	596,471.57 μ s (596.47157 ms)
BIKE-L3	200,405.02 μ s (200.40502 ms)	10,190.90 μ s (10.19090 ms)	161,667.65 μ s (161.66765 ms)

- Kyber768 was the fastest KEM on Raspberry Pi 4, with all operations completing in under 0.12 ms.
- Classic McEliece exhibited the highest latency, with key generation reaching 1907.86 ms and decapsulation up to 596.47 ms.

KEM Benchmark Results: RPI 4 (II)

Raspberry PI 4 - II

ALGORITHM	KEYGEN	ENCAPSULATION	DECAPSULATION
Kyber768	114.55 μ s (0.11455 ms)	114.24 μ s (0.11424 ms)	85.90 μ s (0.08590 ms)
HQC-192	24,297.38 μ s (24.29738 ms)	48,483.15 μ s (48.48315 ms)	72,968.74 μ s (72.96874 ms)
Classic-McEliece-460896	1,779,739.28 μ s (1779.73928 ms)	481.01 μ s (0.48101 ms)	106,641.98 μ s (106.64198 ms)
Classic-McEliece-460896f	481.27 μ s (0.48127 ms)	106,729.86 μ s (106.72986 ms)	599,843.17 μ s (599.84317 ms)
BIKE-L3	200,834.67 μ s (200.83467 ms)	10,201.66 μ s (10.20166 ms)	162,452.61 μ s (162.45261 ms)

- Kyber768 remained the fastest KEM on Raspberry Pi 4-2, with all operations completing in under 0.12 ms.
- Classic McEliece exhibited the highest latency, with key generation reaching 1779.74 ms and decapsulation up to 599.84 ms.

Conclusion - KEM

Kyber768

- Fastest KEM
- Lowest latency on every platform
- The smallest performance difference between constrained and non-constrained devices

HQC-192

- Consistent performance
- Second-best overall KEM
- Alternate candidate

Classic McEliece

- Extremely high key-generation latency ✗
- Not suitable for constrained devices

Signature Benchmark Results: Desktop PC

Desktop PC (Ryzen 9 5950X)

ALGORITHM	KEYPAIR	SIGN	VERIFY
Dilithium3	38.20 μ s (0.03820 ms)	87.22 μ s (0.08722 ms)	35.91 μ s (0.03591 ms)
Falcon-1024	13,876.86 μ s (13.87686 ms)	365.76 μ s (0.36576 ms)	63.59 μ s (0.06359 ms)
SPHINCS+-SHA256-192f-simple	335.66 μ s (0.33566 ms)	9,114.16 μ s (9.11416 ms)	681.29 μ s (0.68129 ms)
SPHINCS+-SHA256-192s-simple	21,209.75 μ s (21.20975 ms)	204,958.24 μ s (204.95824 ms)	296.24 μ s (0.29624 ms)

- Dilithium3 was the fastest signature scheme, completing all operations in under 0.09 ms.
- SPHINCS+-192s had the highest latency, with signing reaching 204.96 ms.

Signature Benchmark Results: Laptop

Laptop (Intel i7-9750H)

ALGORITHM	KEYPAIR	SIGN	VERIFY
Dilithium3	66.83 μ s (0.06683 ms)	166.38 μ s (0.16638 ms)	60.81 μ s (0.06081 ms)
Falcon-1024	26,811.46 μ s (26.81146 ms)	773.34 μ s (0.77334 ms)	128.48 μ s (0.12848 ms)
SPHINCS+-SHA256-192f-simple	618.48 μ s (0.61848 ms)	17,699.89 μ s (17.69989 ms)	1,367.99 μ s (1.36799 ms)
SPHINCS+-SHA256-192s-simple	39,384.89 μ s (39.38489 ms)	397,631.16 μ s (397.63116 ms)	592.89 μ s (0.59289 ms)

- Dilithium3 remained the fastest signature scheme, with all operations completing in under 0.17 ms.
- SPHINCS+-192s showed the highest latency, with signing reaching 397.63 ms.

Signature Benchmark Results: RPI 4 (I)

Raspberry PI 4 - I

ALGORITHM	KEYPAIR	SIGN	VERIFY
Dilithium3	372.94 μs (0.37294 ms)	891.57 μs (0.89157 ms)	307.90 μs (0.30790 ms)
Falcon-1024	62,517.41 μs (62.51741 ms)	1,592.44 μs (1.59244 ms)	207.81 μs (0.20781 ms)
SPHINCS+-SHA256-192f-simple	7,414.37 μs (7.41437 ms)	193,070.54 μs (193.07054 ms)	10,271.99 μs (10.27199 ms)
SPHINCS+-SHA256-192s-simple	475,645.43 μs (475.64543 ms)	4,347,929.33 μs (4347.92933 ms)	3,508.90 μs (3.50890 ms)

- Dilithium3 delivered the best overall performance, with all operations completing in under 0.9 ms.
- SPHINCS+-192s incurred the highest cost, with signing taking 4347.93 ms (4.35 s).

Signature Benchmark Results: RPI 4 (II)

Raspberry PI 4 - II

ALGORITHM	KEYPAIR	SIGN	VERIFY
Dilithium3	372.48 μs (0.37248 ms)	888.45 μs (0.88845 ms)	306.82 μs (0.30682 ms)
Falcon-1024	64,409.84 μs (64.40984 ms)	1,592.87 μs (1.59287 ms)	208.62 μs (0.20862 ms)
SPHINCS+-SHA256-192f-simple	7,434.28 μs (7.43428 ms)	193,806.46 μs (193.80646 ms)	10,617.64 μs (10.61764 ms)
SPHINCS+-SHA256-192s-simple	476,633.05 μs (476.63305 ms)	4,359,148.00 μs (4359.14800 ms)	3,541.26 μs (3.54126 ms)

- Dilithium3 remained the fastest signature scheme, with all operations completing in under 0.9 ms.
- SPHINCS+-192s exhibited the highest latency, with signing requiring 4359.15 ms (4.36 s).

Conclusion - Signature

Dilithium3

- Lowest Keypair latency
- Lowest Sign latency
- Lowest Verify latency
- Best signature algorithm

Falcon-1024

- Fastest verification
- Smallest TLS handshake size
- High keypair latency ✖

SPHINCS+

- Extremely high signing latency ✖
- Poor suitability for constrained IoT devices

Performance Comparison Across Devices

Device	Observation
PC - Ryzen 9 5950X	Lowest latency across KEM and Signature benchmarks
Laptop - Intel i7-9750H	Slower than PC but significantly faster than constrained devices
RPi4-1	Good KEM performance; signature operations incur higher overhead
RPi4-2	Performance nearly identical to RPi4-1

→ **Kyber shows the smallest performance degradation.**

Comparison With Previous Work

KEM ALGORITHMS — COMPARISON WITH PREVIOUS WORK			
Algorithm	Previous Work (Device)	Previous Work Results (KeyGen / Encaps / Decaps)	This Paper (RPI4) (KeyGen / Encaps / Decaps)
Kyber768	Barton et al. [41] (RPI3 B+)	0.89 / 1.08 / 1.32 ms	0.11 / 0.11 / 0.08 ms
HQC-192	Røneid [49] (3.5 GHz CPU)	0.08 / 0.13 / 0.21 ms	24.28 / 48.48 / 72.97 ms
Classic McEliece-460896	Chikouche et al. [45] (Android Mobile)	320313 / 11 / 364 ms	1907.86 / 0.47 / 106.63 ms
Classic McEliece-460896	Røneid [49] (3.5 GHz CPU)	53.95 / 0.03 / 0.07 ms	1907.86 / 0.47 / 106.63 ms
Classic McEliece-460896f (SSF)	Røneid [49] (3.5 GHz CPU)	29.27 / 0.03 / 0.07 ms	0.48 / 106.60 / 596.47 ms
BIKE-L3	Barton et al. [41] (RPI3 B+)	24.50 / 30.30 / 127.00 ms	200.41 / 10.19 / 161.67 ms
BIKE-L3	Røneid [50] (3.5 GHz CPU)	0.44 / 0.12 / 1.65 ms	200.41 / 10.19 / 161.67 ms

- Kyber768 consistently outperformed all other KEMs, achieving the lowest KeyGen, Encapsulation, and Decapsulation latency on constrained devices.
- Performance trends largely agree with previous studies, confirming Kyber as the most suitable KEM for IoT and resource-constrained environments.

SIGNATURE ALGORITHMS — COMPARISON WITH PREVIOUS WORK			
Algorithm	Previous Work (Device)	Previous Work Results (Sign / Verify)	This Paper (RPI4) (Sign / Verify)
Dilithium3	Barton et al. [41] (RPI3 B+)	6.76 / 2.14 ms	0.90 / 0.31 ms
Falcon-1024	Sikeridis et al. [53] (Intel i5 @ 1.7 GHz)	5.22 / 0.05 ms	1.59 / 0.21 ms
SPHINCS+-SHA256-192fs	Barton et al. [41] (RPI3 B+)	785 / 39.7 ms	193.07 / 10.27 ms
SPHINCS+-SHA256-192ss	Barton et al. [41] (RPI3 B+)	21200 / 15.8 ms	4347.93 / 3.51 ms

- Dilithium3 achieved the best overall signature performance, providing the lowest signing and verification latency among all tested schemes.
- Results are consistent with previous work, while SPHINCS+ remained significantly slower, making Dilithium the most practical signature algorithm for constrained devices.

TLS Handshake Analysis

TLS Handshake Performance Comparison - Bytes (Read + Write)

	Kyber768	HQC-192	BIKE-L3
Dilithium	11,694	22,970	15,620
Falcon-1024	7,480	18,758	11,408
SPHINCS+-SHA256-128f	37,388	48,664	41,314

Kyber768 + Falcon1024

- Best TLS Combination
- Handshake Size: 7480 bytes

HQC + SPHINCS+

- Worst TLS Combination
- Handshake Size: 48664 bytes

Important Constraints:

- TLS handshake timing measurements were not reported.
- OpenSSL/Nginx configuration issues prevented successful collection of TLS timing data.

Advantages & Disadvantages

CRYSTALS-Kyber

Advantages:

- Fastest KEM
- Small TLS size

Disadvantage: Less efficient memory usage

HQC

Advantages:

- Balanced performance
- Small public key

Disadvantages: Large TLS handshake size

Classic McEliece

Advantages:

- Very fast encapsulation/decapsulation
- Strong security

Disadvantages: Huge public key, Very slow key generation

BIKE-L3

Advantages:

- Suitable for constrained devices
- Low-cost hardware support

Disadvantages: Expensive key generation and decapsulation

CRYSTALS-Dilithium

Advantages:

- Fastest signature scheme
- IoT-friendly

Disadvantage: Larger signatures than Falcon

Falcon

Advantages:

- Smallest signatures
- Smallest TLS handshake

Disadvantages: Complex implementation, Higher side-channel risk

SPHINCS+

Advantages:

- Very strong security assumptions
- Tiny public/private keys

Disadvantages: Very large signatures, Extremely slow signing

Limitations and Future Work

Limitations:

- TLS handshake timing unavailable
- OpenSSL/Nginx configuration issue
- Only timing metrics measured
- No energy consumption analysis
- No memory usage analysis

Future Work:

- Investigate the unusually high decapsulation latency of Classic McEliece-SSF observed during KEM benchmarking.
- Measure TLS handshake timing in addition to handshake size to provide a complete communication overhead analysis.
- Evaluate additional performance metrics such as memory consumption, CPU utilization, and energy efficiency.
- Benchmark lightweight and optimized PQC implementations designed specifically for constrained IoT devices.

Final Conclusions

Best KEM

CRYSTALS-Kyber (Kyber768)

- Fastest key generation
- Fastest encapsulation
- Fastest decapsulation

Best Signature

CRYSTALS-Dilithium (Dilithium3)

- Fastest keypair
- Fastest signing
- Fastest verification

Best TLS Combination

Kyber768 + Falcon1024

- Smallest handshake size
- 7480 bytes

→ CRYSTALS-Kyber and CRYSTALS-Dilithium are the most practical and efficient post-quantum algorithms for constrained IoT devices, while Falcon provides the most compact TLS handshake. The benchmarking results are consistent with prior literature and support their adoption in resource-constrained environments.